
What the Final Patch Hides: Event-Level Regression Measurement for Coding-Agent Harnesses

Anonymous Author(s)

Affiliation

Address

email

Abstract

1 When a coding agent is graded, only its final patch is examined. We asked what
2 happens in between: does the agent break the repository’s existing tests while it
3 works, and does the final patch show it? We built an instrument that commits the
4 working tree after every mutating tool call and replays the repository’s previously-
5 passing tests at every commit inside the benchmark’s pinned container. We apply
6 it to an unmodified public scaffold: mini-swe-agent 2.4.6, running its own loop,
7 prompts, model layer, budgets, and submission protocol, with only its environ-
8 ment object replaced. On 40 SWE-bench Verified tasks it resolves 34 of 40 with
9 claude-sonnet-4-6 and 33 of 40 with gpt-5.6-sol, ends all 80 runs with a
10 changed tree, and breaks previously-passing test functions in 11 of them — 135
11 counted per run, 100 distinct — of which exactly one is still failing when the
12 official grader reads the final tree. Its own testing is not what hides them: over the
13 window in which each test was broken, no test command ran at all in 6 of the 11
14 breaking edits, and in 4 a run executed the broken test and printed its failure by
15 name. The undercount holds in our own harness under a rollback policy (146 test
16 functions broken across 8 of 47 runs, one still failing) and on a 40-task SWE-bench
17 Live slice, with two caveats we make measurable: rollback leaves clean trees
18 partly because 17 of its 37 runs ship a tree they never changed, and a model-stack
19 difference we measured on Verified does not survive a change of scaffold. The
20 blindness runs both ways — nine of the ten trees whose final state killed the suite
21 outright recorded no timeline event at all. Gating each step on the repository’s
22 own tests instead resolves 65% with no contaminated final tree, and 70% when
23 the gate reads half of those tests and the grader checks the other half. Appendix A
24 catalogues 28 measurement defects met while building the instrument, 23 of which
25 produced plausible numbers rather than errors.

26 1 Introduction

27 Benchmarks for coding agents grade the final patch: SWE-bench [1] and its Verified subset [2]
28 apply the submitted patch, run the repository’s tests, and report whether the target tests pass and the
29 previously passing tests still pass; work on agent-caused regressions counts the previously-passing
30 tests the submitted patch breaks [8].

31 That convention cannot see what happens while the agent works. An agent that breaks an existing
32 test at step three and repairs it at step five submits a clean patch. Two basic questions therefore have
33 no answer: how often do agents break existing behaviour during a run, and how much of it survives
34 to the final patch?

35 We answer both by measuring the timeline instead of the endpoint. Every mutating tool call becomes
36 a commit on a git reference the agent cannot see; after the run, the repository’s previously-passing
37 tests are re-run at every commit inside the benchmark’s pinned container. This is the observation
38 primitive an agent operating system needs before it can build a recovery primitive. Our contributions
39 are:

- 40 1. **An instrument for event-level regression measurement**, applicable to an agent we did not
41 write. Every mutating tool call is recorded as a commit on a git reference the agent cannot
42 see; after the run, the instance’s previously-passing tests are replayed at every recorded
43 state inside the benchmark’s pinned container. Validity is established by controls, a liveness
44 check, a golden check, and a re-scoring control (Section 3).
- 45 2. **The measurement on an unmodified public scaffold**. mini-swe-agent 2.4.6 keeps its own
46 loop, prompts, model layer, budgets, and submission protocol; only its environment object
47 is ours. It resolves 34 and 33 of the same 40 instances under two model families, ends all 80
48 runs with a changed tree, and breaks 135 previously-passing test functions counted per run
49 (100 distinct) across 11 of them, of which one is still failing when the grader reads the tree.
50 In 6 of the 11 breaking edits its own test commands never ran the test it had broken (Section
51 5.1).
- 52 3. **The same measurement across recovery policies, and what the clean tree costs**. The
53 undercount holds in our harness and on SWE-bench Live (Sections 5.2, 5.6), but nearly half
54 of the rollback arm’s runs ship a tree they never changed; a harness-enforced regression gate
55 resolves 65% with no contaminated tree, and 70% when graded on tests the gate never reads
56 (Sections 5.4, 5.5).
- 57 4. **A catalogue of 28 measurement defects** found while building the instrument, classified by
58 mechanism (Appendix A); 23 produced a plausible number rather than an error.

59 All measurements are exploratory, on development slices excluded from future confirmatory study.

60 2 Background and related work

61 **Benchmarks and their validity**. SWE-bench [1] grades a patch by tests that must go from failing to
62 passing and tests that must keep passing; Verified [2] is the human-screened subset, deprecated by its
63 maintainers as a capability metric [46]. Concerns about it have accumulated — contamination and
64 memorisation [6], plausible-but-wrong patches [37], insufficient tests [7] — with rolling [3, 5], private
65 [4], and long-horizon [23] alternatives in response. TDAD [8] measured regressions in *submitted*
66 patches on Verified (6.1% of runs) and reduced them with an impact-analysis map. Process-level
67 benchmarks score intermediate states from logs: RigorBench [35] on 30 curated tasks, SWE-CI [36]
68 across CI iterations, AgentLens [38] naming regression cycles in 10.7% of passing OpenHands runs.
69 We measure the same quantity at every mutating call on standard instances, by executed replay.

70 **Agent scaffolds and recovery**. SWE-agent [9], OpenHands [10], Agentless [11], and mini-swe-agent
71 [12] span the scaffold design space, built on CodeAct [13] and ReAct [14]. Within-episode recovery
72 has been studied as self-reflection [15, 16], progress-gated recovery [17], checkpoint repair [18],
73 and aligned checkpoints [41]; provenance-based recovery is surveyed in [19]. Closest to us, Kim
74 et al. [39] match logged intermediate edits against the gold diff, re-execute the exact matches, and
75 recover agents that reach a gold-identical patch and then destroy it; Gao et al. [40] bind verifier
76 evidence to exact code states; neither counts regressions or compares recovery policies. Running
77 the repository’s tests during a run is deployed practice [43, 44]. Operating-system framings [20]
78 and the branch-context primitive [42] supply scheduling and fork, commit, and abort, but do not
79 observe the tree between actions. The regression problem is program repair’s plausible-versus-correct
80 patch problem [26, 28, 29], where regression tests are the main defence against overfitting [27];
81 trajectory-level evaluation [21, 22] argues resolve rate alone is uninformative. Public leaderboard
82 trajectories record actions but not the tree at each step; the instrument commits the tree.

83 3 Instrument

84 Figure 1 shows the harness and the instrument. **Observational timeline**. After every mutating tool
85 call the working tree is committed to a git reference outside the agent’s view (a private index keeps

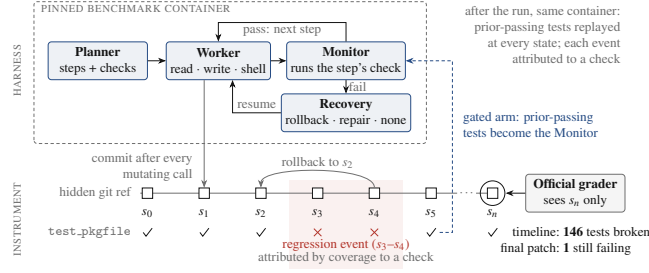


Figure 1: The harness and the instrument. Every mutating tool call commits the tree to a hidden git reference; after the run, the instance’s previously-passing tests are replayed at every state. The test row shows one regressing at s_3 – s_4 and repaired by a rollback to s_2 , which the official grader, reading only s_n , never sees. In the gated arm (dashed), the timeline’s tests serve as the monitor.

86 the agent’s own `git status` unchanged); rollbacks and the end of the run are observations too, and
87 an archived run can be re-measured without re-running the agent.

88 **Exhaustive replay.** For each observation, the instance’s previously-passing (`PASS_TO_PASS`) tests
89 are run inside the pinned container against that observation’s tree; a regression event is a test that
90 passes at one observation and fails at a later one. Every observation is replayed rather than bisected:
91 a recovery policy makes verdicts non-monotone. Replay is scoped to the files holding those tests,
92 which keeps it affordable on Verified (4 to 8 seconds per observation, about 26 CPU-minutes per
93 sweep); Live’s larger oracles cost tens of minutes per instance. Infrastructure failures during replay,
94 and tests that already fail in the unmodified image, are recorded as missing observations rather than
95 as test failures.

96 **Execution environment.** The agent’s tools and the harness’s checks execute inside the same pinned
97 container as the replay, with file changes synchronised to the host tree the timeline records (Appendix
98 A.2).

99 **Validation.** Five checks run before any paid experiment: a negative control, a positive control with
100 injected regressions including recovered ones, a flake screen, an unknown-rate ceiling, and a baseline
101 liveness check. A golden check drives the gold patch through the agent’s real tool path (gold must
102 grade resolved, a null run unresolved); it passed on three repositories per benchmark. A re-scoring
103 control re-measures an archived timeline with no model calls; re-scoring four arms’ bearing runs
104 reproduced their episode counts and probe sets exactly, and supplies the matched-unit columns of
105 Table 1.

106 4 Experimental setup

107 **Benchmark and slice.** SWE-bench Verified, used as a measurement substrate rather than a leader-
108 board: its pinned images and previously-passing test sets make the measurement possible. A
109 40-instance development slice (16 from `django`), stratified and fixed before any measurement, was
110 used throughout and is excluded from any future confirmatory study; the GPT-5.6 stack also ran its
111 first 10 instances as a calibration, and plain rollback ran twice. SWE-bench Live [3], a rolling set
112 built monthly from live repositories, is the second substrate, graded by its own harness whose rules
113 we mirror; its slice was fixed before any outcome by a rule given in Appendix B, and its oracles are
114 about twenty times the Verified slice’s (median 1,189 against 58 previously-passing tests).

115 **Harness.** A planner decomposes the task into steps with verification commands; a worker executes
116 each step with three tools (read, write, shell); a monitor runs the verification, and on failure the
117 recovery policy acts: **rollback** (reset to the last verified checkpoint, retry with feedback), **repair-in-**
118 **place** (retry from the failed tree), or **no recovery** (the monitor’s FAIL is final and the run ends). Runs
119 have a \$4 work-cost cap; exhaustion scores as failure. One instance run under one policy is a cell;
120 each (model stack, policy, substrate) triple is an arm.

121 **Public scaffold.** mini-swe-agent 2.4.6 [12] runs unmodified — its own loop, prompts, model layer,
122 250-step and \$3 billed-cost caps, and submission protocol. Only its environment object is replaced,
123 so every bash command executes in the benchmark’s pinned container and the tree is committed

124 to the hidden reference after each command; the seam is transparent to the agent, whose own
125 `git status` and `git diff` show its work and nothing of the instrument. Six departures from its
126 leaderboard configuration, including a container with no network, are listed in Appendix C; we claim
127 no equivalence to its published resolve rate, and because its budget differs in kind from our cap, no
128 comparison between the two harnesses is tested. A grid census (Appendix C) puts its sampling inside
129 our arms' range — a median of 3 and 2 tree-changing observations per run against our 2 to 5 — while
130 it issues 9 to 15 bash commands per observation, because most of them read.

131 **Models.** Two frontier stacks under identical harness, policy, instances, and caps, named by
132 their API model strings: `claude-opus-4-7` with `claude-sonnet-4-6`, and `gpt-5.6-so1` with
133 `gpt-5.6-terra` (planner, worker), recorded in every run manifest.

134 **Outcomes.** Resolve is the official grader's verdict on the final patch. Events are reported at three
135 levels because a few runs produce most: incidents (observations at which at least one test broke;
136 their count per run is the co-primary endpoint), declared events (one per test function and onset,
137 parametrised variants collapsed), and bearing runs. Because an event and a final-state failure are
138 different objects, the undercount is stated in matched units: distinct test functions broken during a run
139 against those still failing at the end. Final-state contamination is the number of previously-passing
140 tests failing in the graded patch, net of baseline-dead tests. A final tree that kills the suite is graded as
141 failing every test but is a missing observation for event counting, so a contaminated cell need not be a
142 bearing run.

143 **Pre-declaration.** The event unit, the proceed criteria, and the recovery-policy contrast were declared
144 before the relevant data existed, and the contrast's analysis was committed before either comparison
145 arm finished; one amendment preceded unblinding and one grading correction followed it (Section
146 5.4, Appendix B).

147 5 Results

148 5.1 An unmodified public scaffold breaks tests and puts them back

149 mini-swe-agent 2.4.6 ran unmodified on the same 40 Verified instances. With `claude-sonnet-4-6`
150 it resolved 34 of 40 (35 rot-aware) over 2,101 bash commands for \$31.38, four runs ending at its
151 own \$3 cap; with `gpt-5.6-so1`, 33 of 40 (34 rot-aware) over 937 commands for \$12.13, every run
152 ending by its submission protocol. No run of the 80 ended with a tree byte-identical to its start; the
153 four capped runs stopped before the scaffold's submission step, but all four had already changed the
154 tree.

155 The timeline recorded 56 events in 5 of the 40 Claude runs and 79 in 6 of the 40 GPT-5.6 runs, on 8
156 instances, 3 bearing under both models. In matched units those 11 runs broke 135 previously-passing
157 test functions counted per run — 100 distinct, since the two stacks break overlapping sets on the three
158 instances bearing under both — and when the grader read the final tree exactly one was still failing:
159 `test_pkgfile` in `pytest-6197` (Claude 0 of 56; GPT-5.6 1 of 79). The scaffold has no monitor, no
160 checkpoint, and no rollback, so every one of those closures is an ordinary forward edit the agent
161 chose to make.

162 **Would its own testing have caught it?** For each breaking edit we read the scaffold's trajectory over
163 the window in which the test was broken — the breaking command up to, but not including, the repair
164 — and ask whether a test runner ran, whether any run would have executed *this* test, and whether its
165 output showed the failure (Appendix C; coverage is three-valued, *unknown* for runs that stopped early
166 or were truncated). Of the 11 breaking edits, 6 saw no test command run at all while the regression
167 was open; in 4 a run executed the broken test and printed its failure by name; 1 is unknown. What
168 the endpoint misses here is not red output the agent ignored: it is regressions the agent's own test
169 commands never ran.

170 5.2 The same undercount in our harness, and what the clean tree costs

171 Under rollback the Claude stack resolved 13 of 40 (32.5%, exact 95% CI 18.6% to 49.1%) for
172 \$34.24 and the GPT-5.6 stack 13 of 40 for \$8.14; a circuit breaker stopped three of its cells and a
173 synchronisation defect lost two, so 37 were attempted and 35 graded, and one instance is unresolvable
174 under the current official parser (Appendix A.3). The median run makes 5 mutating tool calls.

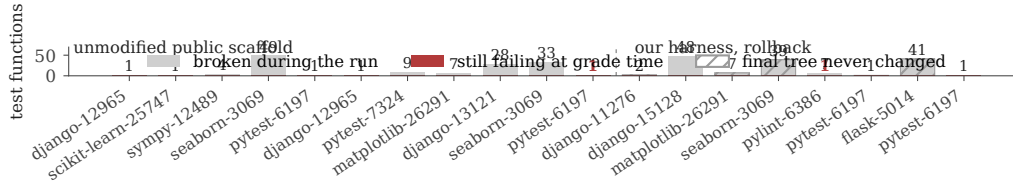


Figure 2: Every run that broke a previously-passing test, in matched units: test functions broken on the timeline against those still failing when the grader reads the final tree (dark). Left, the unmodified public scaffold under both models; right, our rollback arm, where hatched bars are runs whose final tree is byte-identical to their start.

Eight of the 47 GPT-5.6 rollback runs (the sweep and a 10-instance calibration) broke a previously-passing test. Across them 146 test functions were broken counting each run separately (145 distinct — pytest-6197 is bearing in both sweeps), and one — `test_can_read_toml_env_variable` in pylint-6386 — was still failing when the grader read the tree; three storms supply 88% of the 184 declared events, which we report as exposure only.

Not every clean tree is work. Excluding the hidden reference, 17 of the 37 rollback runs end byte-identical to the state they started from, as do 19 of the Claude stack’s 40, 4 of 40 under the gate, and 0 of 40 under repair-in-place. Four of the six bearing sweep runs shipped nothing; restricted to the two that delivered a patch, 1 of 55 broken test functions is still failing. This is why the public-scaffold arm, none of whose 80 runs ends with an unchanged tree, carries the claim.

The blindness runs both ways. A break that stops the suite from being collected opens no timeline event, because a test the parser never mentions is a missing observation rather than a failure, while official grading scores such a patch as failing every test. Every one of the six suite-killed trees in the no-recovery arm, and nine of the ten across all arms, recorded no timeline event at all: the two instruments have complementary blind spots.

5.3 On Verified, a model-stack difference that does not generalise

On the same 40 instances, under the same harness and rollback policy, the Claude stack produced no regression events in 40 runs, the GPT-5.6 stack 140 in 6 of 37 (one-sided Fisher $p = 0.010$). Three controls bound it. Exposure differs about twofold, not 140-fold (147 observations against 218, in opportunity units). The zero is measured, not a detection failure: re-scoring that stack’s earlier canary run reproduces its three events. And it does not survive a change of scaffold — the same `claude-sonnet-4-6` worker under unmodified mini-swe-agent is bearing on 5 of 40 instances with 56 events (Section 5.1), and the difference is not detected on Live. We report it as a measurement on one slice, not a property of either model; model and adapter are confounded.

5.4 Recovery policy: what the clean final tree costs

On the primary endpoint, final-state contamination paired by instance, no recovery was worse than rollback on 9 instances and better on 1 (exact sign test $p = 0.021$); repair-in-place was worse on 5, better on 1 ($p = 0.22$). Incident exposure, the co-primary, did not differ ($p = 0.45$ and 1.0). What the arms cannot show is that the policy *caused* the difference: no recovery ends the run at the monitor’s first FAIL, so its regressions arrive on the last tree-changing observation with nothing after them. Its 19-of-19 persistence is the stopping rule as much as the policy.

Rollback pays for its clean tree in delivered work. It resolved 13 of 40 against 24 and 22, losing nine discordant pairs to repair-in-place and winning one (McNemar $p = 0.021$). The loss is the monitor’s: of the 18 rollback runs that failed, 15 failed at the first step, and 9 of those instances were resolved by an arm that kept work the check had rejected and the grader accepted — and 17 of 37 runs ended with a tree they never changed.

Disclosure. The first unblinding gave $p = 0.375$, because seven cells whose final trees killed the suite had been dropped as ungradable; correcting the grader and re-grading those archived trees gives the 0.022 above — a rule change made after the data were seen (Appendix B).

Table 1: Every arm on both substrates. **empty**: runs whose final tree is byte-identical to their start. **events**: declared (test function, onset) pairs — exposure only, never a denominator. **broken** → **left**: over the bearing runs, distinct previously-passing test functions broken during the run against those still failing when the grader reads the final tree; *marks Live arms with further bearing runs whose failing ids the sidecar did not record*. **contam.**: *graded cells whose final tree fails a previously-passing test, net of baseline-dead* tests* (those already failing in the unmodified image); no recovery’s 9 is six suite-killed trees and three partial, and nine of the ten suite-killed trees across all arms recorded no timeline event at all. The public-scaffold rows ran under mini-swe-agent’s own budget, not our cap, so no comparison between harnesses is tested. Definitions, incidents, and per-row provenance: Appendix C.

substrate	arm	resolved	empty	events	bearing	broken → left	contam.	spend
Verified	mini-swe-agent, Claude	34	0/40	56	5/40	56 → 0	0	\$31.38
Verified	mini-swe-agent, GPT-5.6	33	0/40	79	6/40	79 → 1	1	\$12.13
Verified	GPT-5.6 rollback	13	17/37	140	6/37	104 → 1	1	\$8.14
Verified	Claude rollback	13	19/40	0	0/40	—	1	\$34.24
Verified	GPT-5.6 gated	26	4/40	70	5/40	62 → 0	0	\$10.51
Verified	GPT-5.6 split-gated	28	3/40	14	3/40	14 → 0	0	\$9.40
Verified	GPT-5.6 repair-in-place	24	0/40	84	7/40	83 → 4	5	\$14.73
Verified	GPT-5.6 no recovery	22	1/40	19	3/40	19 → 19	9	\$4.40
Live	GPT-5.6 rollback	0 (2)	25/39	156	6/39	16 → 0*	0	\$10.29
Live	Claude rollback	2 (3)	24/39	49	3/39	43 → 0*	1	\$49.80
Live	GPT-5.6 gated	1 (4)	9/39	280	3/39	267 → 0*	2	\$11.49
Live	GPT-5.6 no recovery	4 (6)	3/39	30	4/39	30 → 30	4	\$8.95

214 5.5 On Verified, regression-gated rollback removes the tradeoff

215 A fourth arm, added after unblinding (exploratory), replaces the planner-written check with the
216 repository’s previously-passing tests, run after every attempt: a step is rejected only if a test that
217 passed at the start fails now. It resolved **26 of 40 (65.0%)**, above rollback’s 13 and the ungated arms’
218 24 and 22, and below the 34 and 33 that unmodified mini-swe-agent reaches on the same instances
219 under a budget differing in kind (Table 1), with **no contaminated final tree** and 4 empty patches
220 against rollback’s 17. Against rollback on 35 shared instances it won 10 discordant pairs and lost 1 (p
221 $= 0.012$). What the gate offers is a guarantee rather than a rate, and it is partly circular: its tests are
222 the benchmark’s own previously-passing set (median ratio 1.00), so a clean tree follows whenever the
223 gate saw the final tree and the grader reads what the gate read. A fifth arm removes the guarantee: the
224 gate reads a deterministic half of the ids (2,278) and never the other half (2,585), on which the grader
225 rules [45]. It resolved **28 of 40 (70.0%)**, indistinguishable from the full gate ($p = 0.73$) and better
226 than rollback (13 to 2, $p = 0.007$), with no held-out failure apart from the parser-capped instance.

227 5.6 Replication on SWE-bench Live

228 Live keeps the undercount and loses both the resolve rates and the stack difference (Table 1). Resolve
229 collapsed for every arm, to at most 6 of 40 even rot-aware against the same four arms’ 13 to 26 on
230 Verified, so the recovery contrast on resolve is uninformative here; the collapse is consistent with
231 contamination of Verified [6, 46], though the substrates differ in oracle size and rot, and 25 of 39
232 rollback runs shipped an unchanged tree. The undercount replicated: GPT-5.6 rollback recorded 156
233 events across 6 of 39 runs and left no contaminated tree, while no recovery left one in each of its
234 4 bearing runs (sign $p = 0.125$). The gate, clean by construction on Verified, left two contaminated
235 trees here (Appendix C); the stack difference did not replicate, the Claude stack being bearing on 3 of
236 39 Live instances against GPT-5.6’s 6 ($p = 0.24$).

237 6 Discussion

238 **Why timeline regressions matter.** A repaired regression is not free: 27% of what the eight Verified
239 bearing runs cost went to work done while a regression was open. The events are the per-step ground
240 truth patch verifiers [24] and process reward models [25] need, and an endpoint-only benchmark
241 cannot tell a run that broke nothing from one that broke 49 tests and put them all back. **Limitations.**
242 Every rate is an estimate on a 40-instance slice with one sweep per arm, and event counts are lower
243 bounds both ways: a suite-killing break is invisible to the timeline, and no-recovery’s persistence is
244 confounded with its stopping rule. Appendix C lists the rest.

References

- [1] C. Jimenez et al. SWE-bench: Can language models resolve real-world GitHub issues? ICLR 2024. arXiv:2310.06770.
- [2] OpenAI. Introducing SWE-bench Verified. OpenAI blog, August 2024. openai.com/index/introducing-swe-bench-verified.
- [3] L. Zhang et al. SWE-bench Goes Live! NeurIPS 2025 Datasets and Benchmarks. arXiv:2505.23419.
- [4] X. Deng et al. SWE-Bench Pro: Can AI agents solve long-horizon software engineering tasks? arXiv:2509.16941, 2025.
- [5] I. Badertdinov et al. SWE-rebench: An automated pipeline for task collection and decontaminated evaluation of software engineering agents. NeurIPS 2025. arXiv:2505.20411.
- [6] S. Liang et al. The SWE-Bench Illusion: When state-of-the-art LLMs remember instead of reason. arXiv:2506.12286, 2025.
- [7] B. Yu et al. UTBoost: Rigorous evaluation of coding agents on SWE-Bench. ACL 2025. arXiv:2506.09289.
- [8] P. Alonso, S. Yovine, and V. A. Braberman. TDAD: Test-driven agentic development: Reducing code regressions in AI coding agents via graph-based impact analysis. arXiv:2603.17973, 2026.
- [9] J. Yang et al. SWE-agent: Agent-computer interfaces enable automated software engineering. NeurIPS 2024. arXiv:2405.15793.
- [10] X. Wang et al. OpenHands: An open platform for AI software developers as generalist agents. ICLR 2025. arXiv:2407.16741.
- [11] C. S. Xia et al. Agentless: Demystifying LLM-based software engineering agents. FSE 2025. arXiv:2407.01489.
- [12] K. Lieret and C. E. Jimenez. mini-swe-agent. Software, 2025. github.com/SWE-agent/mini-swe-agent.
- [13] X. Wang et al. Executable code actions elicit better LLM agents. ICML 2024. arXiv:2402.01030.
- [14] S. Yao et al. ReAct: Synergizing reasoning and acting in language models. ICLR 2023. arXiv:2210.03629.
- [15] N. Shinn et al. Reflexion: Language agents with verbal reinforcement learning. NeurIPS 2023. arXiv:2303.11366.
- [16] A. Madaan et al. Self-Refine: Iterative refinement with self-feedback. NeurIPS 2023. arXiv:2303.17651.
- [17] A. Kadu and A. Krishnan. ReflexGrad: Within-episode failure recovery in LLM agents via progress-gated dual-process routing. arXiv:2511.14584, 2025.
- [18] P. Mazaheri. REPOT: Recoverable program-of-thought via checkpoint repair. arXiv:2605.30052, 2026.
- [19] Y. Wang et al. From agent traces to trust: A survey of evidence tracing and execution provenance in LLM agents. arXiv:2606.04990, 2026.
- [20] K. Mei et al. AIOS: LLM agent operating system. COLM 2025. arXiv:2403.16971.
- [21] S. Kapoor et al. Holistic Agent Leaderboard: The missing infrastructure for AI agent evaluation. arXiv:2510.11977, 2025.
- [22] R. Shu et al. What resolve rate hides: Trajectory structure diagnostics for coding agents. arXiv:2607.06184, 2026.

- 288 [23] T. Le et al. SWE-EVO: Benchmarking coding agents in long-horizon software evolution
289 scenarios. arXiv:2512.18470, 2025.
- 290 [24] M. Raghavendra et al. Agentic rubrics as contextual verifiers for SWE agents. ACL 2026.
291 arXiv:2601.04171.
- 292 [25] M. L. Dihan and M. A. R. Khan. SWE-Shepherd: Advancing PRMs for reinforcing code agents.
293 arXiv:2604.10493, 2026.
- 294 [26] Z. Qi, F. Long, S. Achour, and M. Rinard. An analysis of patch plausibility and correctness for
295 generate-and-validate patch generation systems. ISSTA 2015. doi:10.1145/2771783.2771791.
- 296 [27] Y. Lou et al. When automated program repair meets regression testing: An extensive study on
297 two million patches. ACM TOSEM 33(7), 2024. arXiv:2105.07311.
- 298 [28] Z. Fei et al. Patch correctness assessment: A survey. ACM TOSEM 34(2), 2025.
299 doi:10.1145/3702972.
- 300 [29] H. Ye, M. Martinez, and M. Monperrus. Automated patch assessment for program repair at
301 scale. Empirical Software Engineering 26(2), 2021. doi:10.1007/s10664-020-09920-w.
- 302 [30] SWE-bench issue #601: Test result hijacking via stdout forging in evaluation harness.
303 github.com/SWE-bench/SWE-bench/issues/601, June 2026.
- 304 [31] OpenHands issues #4235 (October 2024) and #7044 (March 2025): Docker and environment
305 errors in SWE-bench instance evaluation. github.com/OpenHands/OpenHands.
- 306 [32] J. Yang et al. SWE-smith: Scaling data for software engineering agents. NeurIPS 2025 Datasets
307 and Benchmarks. arXiv:2504.21798.
- 308 [33] J. Pan et al. Training software engineering agents and verifiers with SWE-Gym. ICML 2025.
309 arXiv:2412.21139.
- 310 [34] S. Liu et al. Context as a tool: Context management for long-horizon SWE-agents. Findings of
311 ACL 2026. arXiv:2512.22087.
- 312 [35] M. B. Madiraju and M. S. P. Madiraju. RigorBench: Benchmarking engineering process
313 discipline in autonomous AI coding agents. arXiv:2606.22678, 2026.
- 314 [36] J. Chen et al. SWE-CI: Evaluating agent capabilities in maintaining codebases via continuous
315 integration. arXiv:2603.03823, 2026.
- 316 [37] Y. Wang, M. Pradel, and Z. Liu. Are "solved issues" in SWE-bench really solved correctly? An
317 empirical study. ICSE 2026. arXiv:2503.15223.
- 318 [38] P. Sahoo et al. AgentLens: Revealing the lucky pass problem in SWE-agent evaluation.
319 arXiv:2605.12925, 2026.
- 320 [39] M. Kim et al. Coherence collapse: Diagnosing why code agents fail after reaching the right
321 code. arXiv:2603.24631, 2026.
- 322 [40] X. Gao, J. Yang, and Q. Yang. Looping is not reliability: State-bound evidence and typed
323 revision contracts for agentic code repair. arXiv:2607.24604, 2026.
- 324 [41] Y. Zhuang et al. AgentRewind: Recoverable execution for long-horizon LLM agents.
325 arXiv:2608.14380, 2026.
- 326 [42] C. Wang and Y. Zheng. Fork, explore, commit: OS primitives for agentic exploration.
327 arXiv:2602.08199, 2026.
- 328 [43] Y. Chen, T. Ahmed, R. Jabbarvand, and M. Hirzel. Can old tests do new tricks for resolving
329 SWE issues? FSE 2026. arXiv:2510.18270.
- 330 [44] P. Gao et al. Trae Agent: An LLM-based agent for software engineering with test-time scaling.
331 arXiv:2507.23370, 2025.

- 332 [45] T. Ahmed, J. Ganhotra, A. Shinnar, and M. Hirzel. Investigating test overfitting on SWE-bench.
333 arXiv:2511.16858, 2025.
- 334 [46] OpenAI. Why SWE-bench Verified no longer measures frontier coding capabilities. OpenAI
335 blog, February 2026. openai.com/index/why-we-no-longer-evaluate-swe-bench-verified.

336 A The measurement defect catalogue

337 A.1 The catalogue by mechanism

338 Each row is a defect found while building the instrument. **S**: it produced a plausible number rather
339 than an error. **G**: it was present while the test suite passed (A4 is the suite). **H**: it was findable only
340 on a clean host. Class A rows depend on the machine the code runs on; class B rows render an
341 infrastructure failure as a measurement; class C rows measure something other than the event as
342 defined; class D rows consume a producer’s output without checking the producer.

#	Defect	Consequence	S	G	H
A1	Shadow commits inherited the machine’s git identity	timeline silently empty on any clean machine	✓	✓	✓
A2	Validation probe invoked bare python	every probe exit 127 on a clean host	✗	✓	✓
A3	Unpinned SDK resolved a different major version	two machines ran different code	✗	✓	✓
A4	Test fixtures verified with bare pytest from PATH	15 tests fail on a clean host as harness defects	✗	—	✓
A5	A benchmark scorer invoked bare python	score 0.0 from a missing interpreter	✓	✓	✓
A6	Agent executed on the host; measurement in the container	26 of 28 zero-step runs; see A.2	✓	✓	✓
B1	Output marker used shell :	a 13/13 run scored as 13 errors	✓	✓	
B2	Results on stderr, markers on stdout	one framework’s results outside the parsed slice	✓	✓	
B3	Probe diffed against a deleted commit	every graded test a hole	✓	✓	
B4	Interrupted mirror clone accepted with zero commits	later instances fail far from the cause	✓	✓	
B5	Container workdir unvalidated	every exec exit 127	✓	✓	
B6	Isolation removed loopback, not only the internet	23.5% of the oracle dead	✓	✓	
343 B7	Provider cache served removed containers	later cells replay against nothing and score clean	✓	✓	
B8	Tree reset reverted image build-time edits	one repository family’s oracle dead	✓	✓	
C1	Bisection assumed monotone verdicts	recovered regressions invisible	✓	✓	
C2	Declared event unit not implemented	rate off by up to 2.7×	✓	✓	
C3	Rollbacks produced no observation	recoveries invisible to the timeline	✓	✓	
C4	"Persists past the step boundary" undefined	one reading excludes the events of interest	✓	✓	
D1	Audit field never populated	0 tool errors reported, always	✓	✓	
D2	Malformed planner output crashed the run	33% of runs lost as task failures	✗	✓	
D3	Dependency install counted as agent work	attribution vacuous	✓	✓	
D4	Join took the first observation, not the last	failures pinned to the wrong tree	✓	✓	
D5	Executed config rebuilt from a name	manifest described a different run	✓	✓	
D6	Git handles never released	sweeps die after ~400 cells	✓	✓	
D7	Console re-walked the tree per run	presented as a dead server	✗	✓	
D8	Detection events lacked the ids attribution joined on	attributed detection always unknown	✓	✓	
D9	Absent coverage rendered as measured silence	unmeasured regressions reported as silent	✓	✓	
D10	Output-capped turns executed	half-written file; a 78-event storm	✓	✓	

344 Totals: 23 of 28 silent; 27 of 28 present under a passing suite; 6 of 28 findable only on a clean host.
345 Ten further defects found after this census closed are described where they arose (Sections 5.2 and
346 5.4, Appendices A.3 and C); the rest will be released with the code.

347 A.2 The defect that invalidated a conclusion

348 After nineteen fixes, every validation check passed, the re-scoring control reproduced archived
349 episode counts exactly, and three pilots (80 runs) measured an event rate far below the pre-declared
350 proceed criterion. We drafted the conclusion the rule directed: the benchmark offered too little
351 opportunity, so switch benchmarks. The conclusion was wrong. The harness ran the agent on the host
352 in a bare source checkout, while every probe and every gate ran inside the pinned container. On the
353 host, `import matplotlib` in that checkout succeeds by importing the uncompiled source tree as a
354 namespace package, and everything downstream fails in ways indistinguishable from an incompetent

agent. Twenty-six of 28 zero-step runs traced to this. The checks had validated the measurement path and never the agent’s execution path. The fix was to route the agent’s tools and checks through the same container as the replay, and to add a per-cell environment parity check that runs before any model call.

359 A.3 Mechanisms in public infrastructure

360 Class A: OpenHands issues #4235 and #7044 [31], environment construction failures against SWE-
361 bench images. Class B: UTBoost’s finding that the held-out oracle is insufficient at leaderboard scale
362 [7], and, on SWE-bench Live, baseline tests that fail in the unmodified image because the calendar
363 passed a deprecation date encoded in the instance. Class C: final-state regression measurement [8].
364 Class D: the SWE-bench grader accepting forged test output on stdout [30], and, at the harness’s
365 current revision, a parser that keys parametrised ids by their full text while the dataset stores them
366 truncated at the first space (the earlier parser’s behaviour), so 16 previously-passing ids of one instance
367 in our slice match no output and every submission to it grades as unresolved.

368 B Pre-declaration and analysis

369 The event unit is one (test function, onset observation) pair with parametrised variants collapsed. The
370 gates for proceeding on a substrate were an event rate of at least 0.30 per run and a bearing fraction
371 of at least 25%. Both stacks failed the conjunction (bearing 0% and 16.2%); the gates were then
372 re-declared per (substrate, model stack) before the contrast was unblinded, and no arm reported here
373 is confirmatory. The contrast’s primary endpoint is final-state contamination, paired by instance, exact
374 two-sided sign test; the co-primary is incident exposure. The analysis script was committed before
375 either comparison arm finished. The regression-gated arm was added after the contrast was unblinded
376 and is exploratory. No multiplicity adjustment was planned or applied. Budget exhaustion is scored
377 as failure. Instances whose baseline oracle records failures in the unmodified image are excluded
378 from resolve-rate comparability claims, and their dead tests are typed as missing observations for
379 event counting.

380 **Compute.** Every run, replay, and grade executed on one 12-vCPU, 22 GB x86 virtual machine
381 running Docker, with no GPUs; the six harness arms on Verified in Table 1 cost \$81.42 in model spend
382 and the two public-scaffold arms \$43.51, plus \$6.87 for the second rollback sweep, the calibration
383 and pilots a comparable amount, and the Live sweeps are reported with their own spend in Table 1.

384 **Slice construction on SWE-bench Live.** Fixed before any outcome: pytest-parsed instances with
385 27 to 3,000 previously-passing tests, at most four per repository, earliest first; the first 40 of 48
386 candidates whose image passed a model-free environment check. No coverage maps are built for
387 Live, so attribution there is unknown by construction.

388 **Attribution (Verified only).** A detected regression is classified three ways: *attributed* if a failing
389 harness check and the broken test both exercise a file the agent changed at that observation (per-
390 instance coverage map at the base commit), *co-occurring* if some check failed while the regression
391 was open without such a link, and *unknown* if the map cannot say. Of the calibration’s 57 raw
392 episodes, 39 were attributed to a failing check, 16 co-occurred with one, and 2 were silent; 48 of the
393 sweep’s 162 had none. The public-scaffold arm has no monitor, so attribution is undefined for it and
394 no silence statistic is reported.

395 **Grading correction between unblindings.** The first unblinding of the recovery-policy contrast
396 gave $p = 0.375$ for the primary endpoint: seven of the eight cells whose final trees made the suite
397 uncollectable (five no-recovery, two repair-in-place) had been dropped as ungradable. Official grading
398 scores such a patch as failing every test, so the most contaminated states had been dropped from the
399 endpoint they bore on most. The grader, not the replay, now distinguishes an environment failure
400 from a patch that kills the suite, and the seven archived trees were re-graded without re-running any
401 agent.

402 C The public-scaffold arm

403 **The seam.** mini-swe-agent’s loop has one place that touches the working tree:
404 `DefaultAgent.execute_actions` calls `env.execute(action)`. We replace the environment

object and nothing else, so the scaffold keeps its own loop, prompts, litellm model layer, step and cost limits, and submission protocol. Our environment runs each bash command inside the benchmark’s pinned container, pulls the resulting tree back to the host the timeline records, and commits an observation when the tree changed. Two properties of the seam were established by measurement rather than assumed. The container’s git state is not advanced by the instrument, so the agent’s own `git status`, `git log`, and `git diff` show its work and nothing else; the first paid pilot, run before this was true, spent eight of twenty commands discovering that its edits had been committed underneath it. And the routed command is wrapped so that a command ending in a here-document terminator or a comment parses as the scaffold’s own `bash -c` would parse it; before that fix, 39 of 40 GPT-5.6 cells hit a shell syntax error, because that model writes every script as a here-document.

Departures from the scaffold’s leaderboard configuration. (1) The measurement container has no network, while its own deployment uses Docker’s default bridge; the unchanged prompt tells the agent it may install tools, and here it cannot. (2) The container’s git history is a single synthetic commit, because the image’s upstream history can contain the fix the task asks for. (3) Resolve is graded on the final tree by the official grader rather than on the scaffold’s submitted patch. (4) Our replay and grading run outside its loop. (5) The two models are reached through the scaffold’s litellm layer, GPT-5.6 through its Responses-API model class, which the chat-completions class cannot serve (it returned three replies with no tool call and the scaffold exited on repeated format errors). (6) Prices come from our verified table rather than litellm’s, which carried a stale entry for one model. For these reasons we claim no equivalence to its published resolve rate.

Sampling grid. Per run, median tree-changing observations and mutated files, with the run and final observations excluded because neither is an opportunity to see a transition: public scaffold 3 and 3 (Claude), 2 and 2 (GPT-5.6); no recovery 2 and 2; repair-in-place 4 and 4; rollback 5 and 7; gate 4 and 4. Observations at which more than one file changed are 0 to 6% in every arm. The scaffold issues 14.8 and 9.1 bash commands per observation against the harness’s one tool call per observation, because most of its commands read rather than write.

The self-test classifier. For each episode we take the window in which the test was broken — the breaking command up to, but not including, the command that repaired it — and label three things over the scaffold’s own trajectory. *Tested*: a test runner ran and its output proves it ran, so a runner that died on an import does not count. *Covered*: one of those runs would have executed this test, decided on the runner’s positional arguments matched at a label boundary (a run of two sibling modules does not cover a test in a third), honouring `-k`, `-m`, `--deselect`, and explicit node ids. *Shown*: a covering run’s output names this test on a line that also carries a failure marker. Coverage and display are three-valued: a run that stopped at an earlier failure under `-x`, or whose output was truncated by a pipe, is *unknown* rather than negative, because calling it negative would assert that the agent ran the test and was not told, from evidence that cannot support it. The classifier’s labels were audited by hand against the raw trajectories of eight bearing cells from an earlier pilot of this arm (run before the two shell defects below were fixed, and therefore not among the runs reported here); the audit disagreed with an earlier version of the classifier on all three totals, and the version reported here reproduces the audit exactly.

The gate’s two contaminated Live trees. One run hit the \$4 cap before the gate saw its final tree; the other passed every check yet fails 13 grader tests, 7 in the file the hidden test patch rewrites.

Further limitations. The public-scaffold arm is one scaffold at one version under two model families; its budget differs in kind from our arms’ (\$3 billed and 250 steps against a \$4 work cost), so the two harnesses are placed side by side and never tested against each other. The self-test measurement rests on 11 breaking edits, one of them coded unknown. Cross-stack comparisons confound model with adapter. The gated arms are post hoc and select tests from benchmark metadata. Live resolve is near floor, and under rollback most Live runs ship nothing, so its recovery contrast is uninformative on resolve.

Reading Table 1. *Baseline-dead* tests already fail in the unmodified image; *rot-aware* resolve counts a run resolved when every target test passes and no failure lies outside that set. On Live, **resolved** is of 40 while **empty** and **bearing** are of the 39 attempted (torch-tune-1806 exceeds the replay budget). **broken** → **left** omits any cell whose suite the grader could not collect, since such a cell would otherwise read as perfect capture; repair-in-place’s row therefore covers six of its seven bearing runs and 83 of its 84 events.

460 **Incidents.** The pre-declared co-primary, per arm, in the order of Table 1: 5, 6, 11, 0, 7, 4, 7, 3, 9, 6, 6,
461 4.

NeurIPS Paper Checklist

1. Claims

Question: Do the main claims made in the abstract and introduction accurately reflect the paper’s contributions and scope?

Answer: [Yes]

Justification: The abstract and Section 1 state the scope (a pilot on a 40-instance slice per substrate, one sweep per arm), the four contributions, and that all measurements are exploratory; Section 6 lists the limitations that bound them.

Guidelines:

- The answer [N/A] means that the abstract and introduction do not include the claims made in the paper.
- The abstract and/or introduction should clearly state the claims made, including the contributions made in the paper and important assumptions and limitations. A [No] or [N/A] answer to this question will not be perceived well by the reviewers.
- The claims made should match theoretical and experimental results, and reflect how much the results can be expected to generalize to other settings.
- It is fine to include aspirational goals as motivation as long as it is clear that these goals are not attained by the paper.

2. Limitations

Question: Does the paper discuss the limitations of the work performed by the authors?

Answer: [Yes]

Justification: Section 6 covers slice size, single sweeps, the adapter confound of the cross-stack comparison, the post hoc gated arms and their selection leakage, trajectory length relative to public scaffolds, Live resolve near floor, and event counts as lower bounds; Section 5.4 discloses the grading correction made after unblinding.

Guidelines:

- The answer [N/A] means that the paper has no limitation while the answer [No] means that the paper has limitations, but those are not discussed in the paper.
- The authors are encouraged to create a separate “Limitations” section in their paper.
- The paper should point out any strong assumptions and how robust the results are to violations of these assumptions (e.g., independence assumptions, noiseless settings, model well-specification, asymptotic approximations only holding locally). The authors should reflect on how these assumptions might be violated in practice and what the implications would be.
- The authors should reflect on the scope of the claims made, e.g., if the approach was only tested on a few datasets or with a few runs. In general, empirical results often depend on implicit assumptions, which should be articulated.
- The authors should reflect on the factors that influence the performance of the approach. For example, a facial recognition algorithm may perform poorly when image resolution is low or images are taken in low lighting. Or a speech-to-text system might not be used reliably to provide closed captions for online lectures because it fails to handle technical jargon.
- The authors should discuss the computational efficiency of the proposed algorithms and how they scale with dataset size.
- If applicable, the authors should discuss possible limitations of their approach to address problems of privacy and fairness.
- While the authors might fear that complete honesty about limitations might be used by reviewers as grounds for rejection, a worse outcome might be that reviewers discover limitations that aren’t acknowledged in the paper. The authors should use their best judgment and recognize that individual actions in favor of transparency play an important role in developing norms that preserve the integrity of the community. Reviewers will be specifically instructed to not penalize honesty concerning limitations.

3. Theory assumptions and proofs

Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

Answer: [N/A]

Justification: The paper contains no theoretical results.

Guidelines:

- The answer [N/A] means that the paper does not include theoretical results.
- All the theorems, formulas, and proofs in the paper should be numbered and cross-referenced.
- All assumptions should be clearly stated or referenced in the statement of any theorems.
- The proofs can either appear in the main paper or the supplemental material, but if they appear in the supplemental material, the authors are encouraged to provide a short proof sketch to provide intuition.
- Inversely, any informal proof provided in the core of the paper should be complemented by formal proofs provided in appendix or supplemental material.
- Theorems and Lemmas that the proof relies upon should be properly referenced.

4. Experimental result reproducibility

Question: Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper (regardless of whether the code and data are provided or not)?

Answer: [Yes]

Justification: Section 3 specifies the instrument (commit per mutating call, replay of previously-passing tests in the pinned container, attribution, validation gates), Section 4 the substrates, slice rules, harness, tools, model identifiers, cost caps, outcomes and pre-declaration, and Appendix B the analysis plan; the defect catalogue (Appendix A) records the measurement pitfalls a replication must avoid.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- If the paper includes experiments, a [No] answer to this question will not be perceived well by the reviewers: Making the paper reproducible is important, regardless of whether the code and data are provided or not.
- If the contribution is a dataset and/or model, the authors should describe the steps taken to make their results reproducible or verifiable.
- Depending on the contribution, reproducibility can be accomplished in various ways. For example, if the contribution is a novel architecture, describing the architecture fully might suffice, or if the contribution is a specific model and empirical evaluation, it may be necessary to either make it possible for others to replicate the model with the same dataset, or provide access to the model. In general, releasing code and data is often one good way to accomplish this, but reproducibility can also be provided via detailed instructions for how to replicate the results, access to a hosted model (e.g., in the case of a large language model), releasing of a model checkpoint, or other means that are appropriate to the research performed.
- While NeurIPS does not require releasing code, the conference does require all submissions to provide some reasonable avenue for reproducibility, which may depend on the nature of the contribution. For example
 - (a) If the contribution is primarily a new algorithm, the paper should make it clear how to reproduce that algorithm.
 - (b) If the contribution is primarily a new model architecture, the paper should describe the architecture clearly and fully.
 - (c) If the contribution is a new model (e.g., a large language model), then there should either be a way to access this model for reproducing the results or a way to reproduce the model (e.g., with an open-source dataset or instructions for how to construct the dataset).

- (d) We recognize that reproducibility may be tricky in some cases, in which case authors are welcome to describe the particular way they provide for reproducibility. In the case of closed-source models, it may be that access to the model is limited in some way (e.g., to registered users), but it should be possible for other researchers to have some path to reproducing or verifying the results.

5. Open access to data and code

Question: Does the paper provide open access to the data and code, with sufficient instructions to faithfully reproduce the main experimental results, as described in supplemental material?

Answer: [No]

Justification: The benchmark data are public (SWE-bench Verified, SWE-bench Live). The instrument, the sweep driver, the analysis scripts and the archived timelines will be released with the camera-ready version; they are not attached to the anonymous submission.

Guidelines:

- The answer [N/A] means that paper does not include experiments requiring code.
- Please see the NeurIPS code and data submission guidelines (<https://neurips.cc/public/guides/CodeSubmissionPolicy>) for more details.
- While we encourage the release of code and data, we understand that this might not be possible, so [No] is an acceptable answer. Papers cannot be rejected simply for not including code, unless this is central to the contribution (e.g., for a new open-source benchmark).
- The instructions should contain the exact command and environment needed to run to reproduce the results. See the NeurIPS code and data submission guidelines (<https://neurips.cc/public/guides/CodeSubmissionPolicy>) for more details.
- The authors should provide instructions on data access and preparation, including how to access the raw data, preprocessed data, intermediate data, and generated data, etc.
- The authors should provide scripts to reproduce all experimental results for the new proposed method and baselines. If only a subset of experiments are reproducible, they should state which ones are omitted from the script and why.
- At submission time, to preserve anonymity, the authors should release anonymized versions (if applicable).
- Providing as much information as possible in supplemental material (appended to the paper) is recommended, but including URLs to data and code is permitted.

6. Experimental setting/details

Question: Does the paper specify all the training and test details (e.g., data splits, hyperparameters, how they were chosen, type of optimizer) necessary to understand the results?

Answer: [Yes]

Justification: No models are trained. Section 4 gives the slices and how they were fixed, the harness and its three recovery policies (the two regression-gated variants are specified in Section 5.6), the exact model strings, the per-run cost cap, the outcome definitions, and the pre-declared analysis.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The experimental setting should be presented in the core of the paper to a level of detail that is necessary to appreciate the results and make sense of them.
- The full details can be provided either with the code, in appendix, or as supplemental material.

7. Experiment statistical significance

Question: Does the paper report error bars suitably and correctly defined or other appropriate information about the statistical significance of the experiments?

Answer: [Yes]

Justification: Resolve and bearing fractions carry exact (Clopper–Pearson) 95% intervals; paired comparisons use exact sign tests and exact McNemar tests, the cross-stack comparison a one-sided Fisher exact test with its sensitivity to reassigned runs stated; the tests reported and the absence of multiplicity adjustment are declared (Section 5.6, Appendix B). Run-to-run variability is measured only for plain rollback (Section 5.3), and intervals treat instances as independent although 16 of the 40 Verified instances come from django.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The authors should answer [Yes] if the results are accompanied by error bars, confidence intervals, or statistical significance tests, at least for the experiments that support the main claims of the paper.
- The factors of variability that the error bars are capturing should be clearly stated (for example, train/test split, initialization, random drawing of some parameter, or overall run with given experimental conditions).
- The method for calculating the error bars should be explained (closed form formula, call to a library function, bootstrap, etc.)
- The assumptions made should be given (e.g., Normally distributed errors).
- It should be clear whether the error bar is the standard deviation or the standard error of the mean.
- It is OK to report 1-sigma error bars, but one should state it. The authors should preferably report a 2-sigma error bar than state that they have a 96% CI, if the hypothesis of Normality of errors is not verified.
- For asymmetric distributions, the authors should be careful not to show in tables or figures symmetric error bars that would yield results that are out of range (e.g., negative error rates).
- If error bars are reported in tables or plots, the authors should explain in the text how they were calculated and reference the corresponding figures or tables in the text.

8. Experiments compute resources

Question: For each experiment, does the paper provide sufficient information on the computer resources (type of compute workers, memory, time of execution) needed to reproduce the experiments?

Answer: [Yes]

Justification: Section 3 gives replay cost per observation and per sweep; Table 1 gives model spend per arm and substrate; Appendix B states the machine (one 12-vCPU, 22 GB x86 virtual machine running Docker, no GPUs) and the total model spend including pilots.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The paper should indicate the type of compute workers CPU or GPU, internal cluster, or cloud provider, including relevant memory and storage.
- The paper should provide the amount of compute required for each of the individual experimental runs as well as estimate the total compute.
- The paper should disclose whether the full research project required more compute than the experiments reported in the paper (e.g., preliminary or failed experiments that didn't make it into the paper).

9. Code of ethics

Question: Does the research conducted in the paper conform, in every respect, with the NeurIPS Code of Ethics <https://neurips.cc/public/EthicsGuidelines>?

Answer: [Yes]

Justification: The work measures software agents on public benchmarks; no human subjects, personal data, or high-risk assets are involved, and the submission is anonymized.

Guidelines:

- The answer [N/A] means that the authors have not reviewed the NeurIPS Code of Ethics.

- If the authors answer [No], they should explain the special circumstances that require a deviation from the Code of Ethics.
- The authors should make sure to preserve anonymity (e.g., if there is a special consideration due to laws or regulations in their jurisdiction).

10. Broader impacts

Question: Does the paper discuss both potential positive societal impacts and negative societal impacts of the work performed?

Answer: [No]

Justification: The contribution is an evaluation instrument and measurements of it. The plausible impact is positive (agents that break less existing behaviour); we see no direct path to a negative application, and the paper does not discuss societal impact.

Guidelines:

- The answer [N/A] means that there is no societal impact of the work performed.
- If the authors answer [N/A] or [No], they should explain why their work has no societal impact or why the paper does not address societal impact.
- Examples of negative societal impacts include potential malicious or unintended uses (e.g., disinformation, generating fake profiles, surveillance), fairness considerations (e.g., deployment of technologies that could make decisions that unfairly impact specific groups), privacy considerations, and security considerations.
- The conference expects that many papers will be foundational research and not tied to particular applications, let alone deployments. However, if there is a direct path to any negative applications, the authors should point it out. For example, it is legitimate to point out that an improvement in the quality of generative models could be used to generate Deepfakes for disinformation. On the other hand, it is not needed to point out that a generic algorithm for optimizing neural networks could enable people to train models that generate Deepfakes faster.
- The authors should consider possible harms that could arise when the technology is being used as intended and functioning correctly, harms that could arise when the technology is being used as intended but gives incorrect results, and harms following from (intentional or unintentional) misuse of the technology.
- If there are negative societal impacts, the authors could also discuss possible mitigation strategies (e.g., gated release of models, providing defenses in addition to attacks, mechanisms for monitoring misuse, mechanisms to monitor how a system learns from feedback over time, improving the efficiency and accessibility of ML).

11. Safeguards

Question: Does the paper describe safeguards that have been put in place for responsible release of data or models that have a high risk for misuse (e.g., pre-trained language models, image generators, or scraped datasets)?

Answer: [N/A]

Justification: No models or scraped datasets are released.

Guidelines:

- The answer [N/A] means that the paper poses no such risks.
- Released models that have a high risk for misuse or dual-use should be released with necessary safeguards to allow for controlled use of the model, for example by requiring that users adhere to usage guidelines or restrictions to access the model or implementing safety filters.
- Datasets that have been scraped from the Internet could pose safety risks. The authors should describe how they avoided releasing unsafe images.
- We recognize that providing effective safeguards is challenging, and many papers do not require this, but we encourage authors to take this into account and make a best faith effort.

12. Licenses for existing assets

Question: Are the creators or original owners of assets (e.g., code, data, models), used in the paper, properly credited and are the license and terms of use explicitly mentioned and properly respected?

Answer: [\[Yes\]](#)

Justification: SWE-bench, SWE-bench Verified and SWE-bench Live are cited [1, 2, 3] and used under their MIT licenses and published container images; the models are accessed through their providers' APIs under the providers' terms; every cited artefact appears in the references.

Guidelines:

- The answer [\[N/A\]](#) means that the paper does not use existing assets.
- The authors should cite the original paper that produced the code package or dataset.
- The authors should state which version of the asset is used and, if possible, include a URL.
- The name of the license (e.g., CC-BY 4.0) should be included for each asset.
- For scraped data from a particular source (e.g., website), the copyright and terms of service of that source should be provided.
- If assets are released, the license, copyright information, and terms of use in the package should be provided. For popular datasets, paperswithcode.com/datasets has curated licenses for some datasets. Their licensing guide can help determine the license of a dataset.
- For existing datasets that are re-packaged, both the original license and the license of the derived asset (if it has changed) should be provided.
- If this information is not available online, the authors are encouraged to reach out to the asset's creators.

13. New assets

Question: Are new assets introduced in the paper well documented and is the documentation provided alongside the assets?

Answer: [\[No\]](#)

Justification: The instrument and the archived timelines are not released with the anonymous submission; they will be released, documented, with the camera-ready version.

Guidelines:

- The answer [\[N/A\]](#) means that the paper does not release new assets.
- Researchers should communicate the details of the dataset/code/model as part of their submissions via structured templates. This includes details about training, license, limitations, etc.
- The paper should discuss whether and how consent was obtained from people whose asset is used.
- At submission time, remember to anonymize your assets (if applicable). You can either create an anonymized URL or include an anonymized zip file.

14. Crowdsourcing and research with human subjects

Question: For crowdsourcing experiments and research with human subjects, does the paper include the full text of instructions given to participants and screenshots, if applicable, as well as details about compensation (if any)?

Answer: [\[N/A\]](#)

Justification: No crowdsourcing or human-subject research.

Guidelines:

- The answer [\[N/A\]](#) means that the paper does not involve crowdsourcing nor research with human subjects.
- Including this information in the supplemental material is fine, but if the main contribution of the paper involves human subjects, then as much detail as possible should be included in the main paper.

774 • According to the NeurIPS Code of Ethics, workers involved in data collection, curation,
775 or other labor should be paid at least the minimum wage in the country of the data
776 collector.

777 **15. Institutional review board (IRB) approvals or equivalent for research with human**
778 **subjects**

779 Question: Does the paper describe potential risks incurred by study participants, whether
780 such risks were disclosed to the subjects, and whether Institutional Review Board (IRB)
781 approvals (or an equivalent approval/review based on the requirements of your country or
782 institution) were obtained?

783 Answer: [N/A]

784 Justification: No human subjects.

785 Guidelines:

786 • The answer [N/A] means that the paper does not involve crowdsourcing nor research
787 with human subjects.

788 • Depending on the country in which research is conducted, IRB approval (or equivalent)
789 may be required for any human subjects research. If you obtained IRB approval, you
790 should clearly state this in the paper.

791 • We recognize that the procedures for this may vary significantly between institutions
792 and locations, and we expect authors to adhere to the NeurIPS Code of Ethics and the
793 guidelines for their institution.

794 • For initial submissions, do not include any information that would break anonymity (if
795 applicable), such as the institution conducting the review.

796 **16. Declaration of LLM usage**

797 Question: Does the paper describe the usage of LLMs if it is an important, original, or
798 non-standard component of the core methods in this research? Note that if the LLM is used
799 only for writing, editing, or formatting purposes and does *not* impact the core methodology,
800 scientific rigor, or originality of the research, declaration is not required.

801 Answer: [Yes]

802 Justification: LLMs are the agents under measurement: the planner, worker and monitor
803 models are named by their API identifiers in Section 4, and their roles in the harness are
804 described in Sections 3 and 4.

805 Guidelines:

806 • The answer [N/A] means that the core method development in this research does not
807 involve LLMs as any important, original, or non-standard components.

808 • Please refer to our LLM policy in the NeurIPS handbook for what should or should not
809 be described.